SMART LIBRARY REQUEST WORKFLOW IN SERVICENOW

PROJECT DOCUMENTATION

1. INTRODUCTION:

Project Title: Smart Library Request Workflow in Service Now

Team Member: K. KARTHIKA DEVI

Team Member: T. RASIKA

Team Member: M. HEMALATHA

Team Member: M. ROSHINI

Team Member: E. MAHESHYAZHINI

2. PROJECT OVERVIEW:

Purpose:

The **Smart Library Request Workflow** is designed to automate the process of requesting, approving, and managing library resources using the ServiceNow platform. The system reduces manual work and improves efficiency by providing a centralized workflow for handling book requests, approvals, and notifications.

Goals:

- Automate library book request processes
- Provide transparency for users and administrators
- Reduce manual paperwork
- Improve request tracking and management

Key Features:

- User login and authentication
- Book request submission
- Automated approval workflow
- Admin dashboard for request management
- Notification system for request status

- Database management for books and users

3. ARCHITECTURE:

Frontend (React):

- The frontend is developed using **React.js**, which provides a responsive and interactive user interface.

Components:

- Login Page
- User Dashboard
- Book Request Form
- Request Status Page
- Admin Panel

Technologies Used:

- React.js
- HTML5
- CSS3
- JavaScript
- Axios (API calls)

Backend (Node.js + Express.js):

- The backend is developed using **Node.js and Express.js** to handle API requests and business logic.

Responsibilities

- API handling
- Request processing
- Authentication management
- Communication with the database

Key Modules

- Authentication module

- Request management module
- Notification module
- Admin management module

Database (MongoDB)

- MongoDB stores all application data including users, books, and request information.

Collections

lection	scription
rs	res user details
ks	res book information
uests	res book request data
nin	res admin details

Example Schema

User Schema

- UserId
- Name
- Email
- Password
- Role

Book Schema

- BookId
- Title
- Author
- Category
- Availability

Request Schema

- RequestId

- UserId
- BookId
- RequestDate
- Status

4. SETUP INSTRUCTIONS:

Prerequisites

Install the following software:

- Node.js
- MongoDB
- Git
- VS Code
- Web Browser (Chrome recommended)

Installation Steps

Step 1 – Clone the repository

- git clone https://github.com/username/smart-library-workflow.git

Step 2 – Navigate to project folder

- cd smart-library-workflow

Step 3 – Install dependencies

Client:

- cd client
- npm install

Server:

- cd server
- npm install

Step 4 – Setup environment variables

Create a **.env file**

- PORT=5000
- MONGO_URI=your_mongodb_connection
- JWT_SECRET=your_secret_key

5. FOLDER STRUCTURE:

Client (React Frontend)

```
client/
│
├── public/
├── src/
│   ├── components/
│   │   ├── Login.js
│   │   ├── Dashboard.js
│   │   ├── RequestForm.js
│   │   └── AdminPanel.js
│   │
│   ├── pages/
│   ├── services/
│   ├── App.js
│   └── index.js
│
└── package.json
```

Server (Node.js Backend)

```
server/
│
├── config/
│   └── db.js
│
├── models/
```

```
│   ├── User.js
│   ├── Book.js
│   └── Request.js
│
├── routes/
│   ├── authRoutes.js
│   ├── bookRoutes.js
│   └── requestRoutes.js
│
├── controllers/
│
├── middleware/
│
├── server.js
└── package.json
```

6. RUNNING THE APPLICATION:

Start Backend

- Navigate to the server folder:
- cd server
- npm start
- Server will run on:
- http://localhost:5000

Start Frontend

Navigate to the client folder:

- cd client
- npm start

Frontend will run on:

- http://localhost:3000

7. API DOCUMENTATION:

1. User Login

Endpoint

- POST /api/auth/login

Request Body

```
{
  "email": "user@email.com",
  "password": "123456"
}
```

Response

```
{
  "message": "Login successful",
  "token": "jwt_token_here"
}
```

2. Request Book

Endpoint

POST /api/request/book

Request Body

```
{
  "userId": "123",
  "bookId": "456"
}
```

3. Get Request Status

Endpoint

- GET /api/request/status/:id

4. Admin Approve Request

- PUT /api/request/approve/:id

8. AUTHENTICATION:

- Authentication is handled using **JWT (JSON Web Token)**.

Process

1. User logs in with credentials.

2. Server validates user.
3. JWT token is generated.
4. Token is sent to the client.
5. Client includes token in future API requests.

Example Header

- Authorization: Bearer <token>

Benefits

- Secure communication
- Stateless authentication
- Easy integration with APIs

9. USER INTERFACE:

Main UI Screens

1. Login Page
2. User Dashboard
3. Book Request Form
4. Request Status Page
5. Admin Approval Panel

UI Features

1. Simple and user-friendly interface
2. Responsive design
3. Real-time request updates
4. Clear request status display

10. TESTING:

Testing Strategy

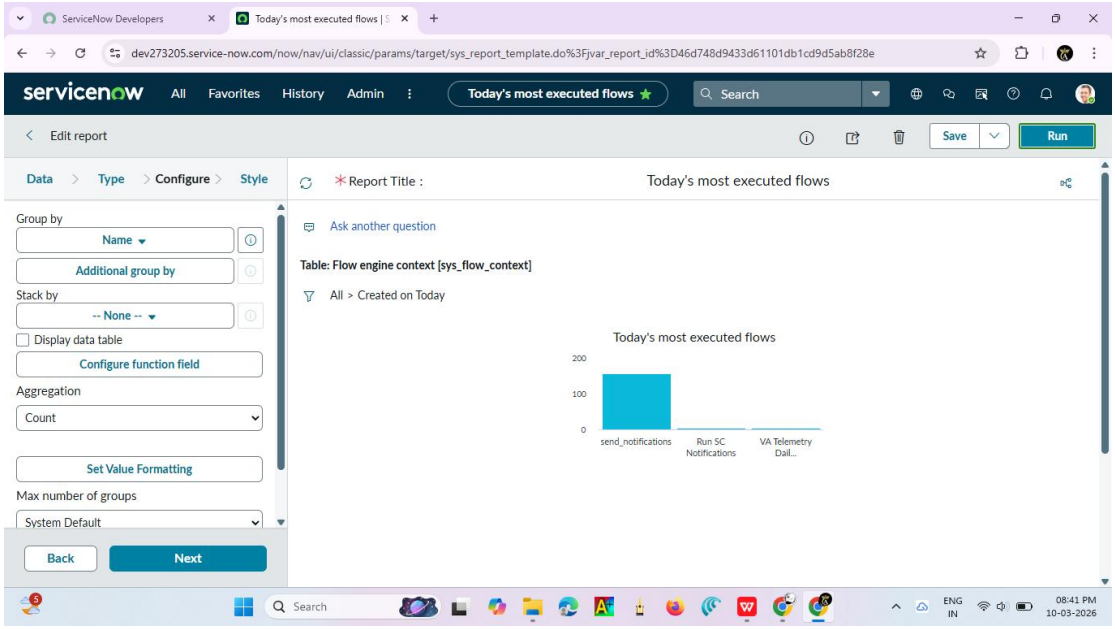
- The system is tested using multiple testing methods to ensure reliability.

Types of Testing

Tools Used

ting Type	scription

ting Type	scription
t Testing	ting individual modules
gration Testing	ting interaction between modules
tem Testing	ting the complete system
r Acceptance Testing	uring system meets user requirements

- Postman (API testing)
- Jest (Unit testing)
- Manual UI testing

FINAL OUTPUT: